



BookStore Project Documentation

PREPARED BY

Alyeldin Khairy

Mustafa Rabea

1. Project Overview

BookStore is a simple yet functional e-commerce website designed for a bookstore, enabling users to browse books, explore author profiles, manage a shopping cart, complete checkout, and handle basic account registration and login. The primary focus is on helping literature enthusiasts discover and purchase books with an intuitive and responsive interface. The target audience includes book lovers, particularly readers in regions such as Egypt (reflected in the use of EGP currency and Cairo-based references). The main goal of the project is to deliver a clean, user-friendly experience for browsing, searching, and buying books while demonstrating modern front-end development practices. Currently, the project operates as a front-end focused prototype with Supabase integration for data retrieval. It does not include a full custom backend server and relies heavily on client-side logic and browser storage.

2. Architecture

The application follows a client-heavy architecture with multiple static HTML pages dynamically enhanced through JavaScript. It behaves somewhat like a single-page application in terms of interactivity, although each major section lives in its own HTML file. The front-end is built using HTML5 for structure, custom CSS combined with Bootstrap for responsive styling, and vanilla JavaScript for all dynamic behavior and business logic.

Data persistence is handled in two layers. Supabase serves as the Backend-as-a-Service (BaaS), providing a PostgreSQL database accessed via REST API calls for reading book and author information. All persistent content (books, authors, images, descriptions) is fetched from Supabase. User accounts and shopping cart data, however, are stored entirely in the browser using LocalStorage. This keeps the project simple and serverless but limits security and scalability.

Data flows from user interaction → JavaScript fetching from Supabase → dynamic rendering of content (product lists, author cards, etc.). Cart and user session data are maintained locally and persist across page navigations. The application is suitable for static hosting platforms such as GitHub Pages, Netlify, or Vercel, with Supabase handling the database backend securely through API keys.

Security remains basic: Supabase API keys are hardcoded in client-side code (not recommended for production), user data and passwords are stored in plain text in LocalStorage, and there is no server-side authentication or password hashing. For real-world usage, a proper backend with JWT authentication and secure storage would be necessary.

3. Approach and Methodology

The development followed an iterative, agile-inspired process. The project began with static HTML page skeletons, followed by styling with Bootstrap and custom CSS, and then progressive addition of JavaScript functionality. Core features were prioritized in this order: browsing books and authors → cart management → user authentication → checkout simulation.

The codebase was kept modular by separating each major page into its own HTML file and extracting shared components (header and footer) into JavaScript files that inject content dynamically on page load. All logic remains client-side with no server-side rendering or processing. Asynchronous data fetching is handled using the modern Fetch API with `async/await` syntax. User feedback is improved through SweetAlert2 modals instead of plain browser alerts.

Key technical decisions included:

- Using LocalStorage for rapid prototyping of user sessions and cart instead of implementing full authentication flows
- Choosing Supabase over building a custom backend to eliminate server management during development
- Building with vanilla JavaScript (no React, Vue, or other frameworks) to keep the project lightweight and focused on core web technologies
- Relying on Bootstrap's built-in form validation combined with custom JavaScript checks for password strength, email uniqueness, and matching fields

Challenges addressed during development included managing asynchronous data loading, maintaining cart state across pages, implementing basic form validation, and creating a consistent navigation experience without a routing library.

Future improvements could include integrating a real payment gateway (Stripe/Paymob), adding server-side authentication, implementing full-text search optimization, creating an admin interface for managing books and authors, and migrating to a modern JavaScript framework for better state management and maintainability.

4. Features and Pages

The application consists of the following main pages, each serving a distinct purpose:

- **Home (index.html):** Features a carousel slider, hero section, and feature sections highlighting the store's mission and services.
- **About (about.html):** Tells the story of the bookstore, its values, and includes a call-to-action to start shopping.
- **Products (products.html):** Displays all books with search input and author-based filtering; fetches data from Supabase and supports adding items to cart (requires login).
- **Authors (authors.html):** Shows author profile cards with photos, bios, age, and links to view all books by that author.
- **Contact (contact.html):** Provides a contact form along with physical location, email, and phone information.
- **Login (login.html):** Simple email/password form that validates against LocalStorage user records.
- **Signup (signup.html):** Registration form with name, email, password, and confirmation fields; performs basic validation and stores new users in LocalStorage.
- **Cart (cart.html):** Displays cart contents from LocalStorage with quantity controls, item removal, and order summary.
- **Checkout (checkout.html):** Collects billing details, offers cash or credit payment simulation, shows order summary, and clears cart upon successful submission.
- **Product Details (product-details.html):** Shows detailed information for a single book including description, price, quantity selector, and add-to-cart functionality.
- **Profile (profile.html):** Currently a placeholder page with no implemented functionality.

Shared components such as the header (with navigation and conditional cart/login links) and footer are injected dynamically via JavaScript on every page.

5. Technologies Used

- **Front-End:**
 - HTML5 – page structure
 - CSS – custom styling (style.css) + Bootstrap 5.3 – responsive layout and components
 - Vanilla JavaScript – all interactivity, data fetching, DOM manipulation, and event handling
- **Libraries & Services:**
 - Bootstrap 5.3 – grid system, cards, forms, modals, icons
 - SweetAlert2 – beautiful user confirmation and error messages
 - Google Fonts (Roboto + Lato) – typography
 - Supabase – cloud PostgreSQL database and REST API for books & authors
- **Browser Storage:**
 - localStorage – user accounts (email, password, name) and shopping cart contents

No build tools, bundlers, or package managers are used — the project remains plain static files that can be served directly.

6. How It Works

1. Users register or log in → credentials are stored/checked in localStorage.
2. Browsing products or authors triggers asynchronous fetches from Supabase.
3. Clicking “Add to Cart” checks if the user is logged in → if yes, the item is added/updated in the localStorage cart.
4. The cart page reads localStorage, shows items, allows quantity changes and removal.
5. Checkout collects billing info → on valid submission, simulates order success, shows thank-you message, and clears the cart.

7. Setup and Running

To run the project locally, simply open any HTML file (preferably index.html) in a modern browser. No installation or server is required. For Supabase integration to work, the provided anonymous API key must remain valid and point to a correctly configured Supabase project containing the books and authors tables with appropriate columns and relationships.